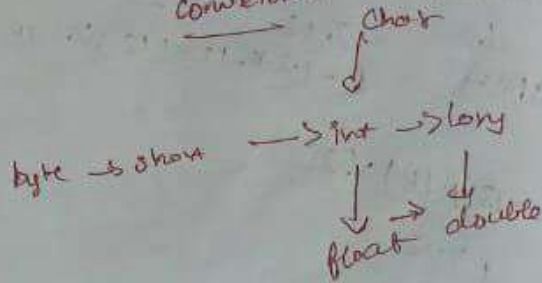


Numeric Type Conversion



Conversion shows more lossless conversion

lossless conversions → not loss values when char, date, etc.

10
int → 10L
lossy

not loss values

10.3f → Double → 10.3

lossy conversion

100 → 100

int
12 → 0.3 loss

float
12.3
lossy
not direct allowed
any way

Casting



Force to loss value

Implicit

↓
Changing
Data
Automat

Wider

Explicit

(cast)

→ Forced
to loss
value

Narrower

Binary

operation (Arithmetic)

double → double

float → float

long → long

↓
or add int

Make sure C1

byte b = 256

forced to 0

boolean X
var not → true
an X

Assignment

a = 12

assign the value

from r/r

left

a = a + 2

var
12

var

short int

=
* =
/ =
% =

byte b = 256;

wont lossy

byte b = 0;

b += 256; wor

↓
a = b = (int) b + 256;

↓
coerced

Increment | Decrement

++ --
 ↓ ↓
 add 1 subtract 1

Where we put change the game >
 pre

++a
 ↓
 increases and then output
 a = 10;
 x = ++a
 x = 11

a = 10; post
 x = a++;
 x = 10
 a = 11

x = 100;

y = 20;

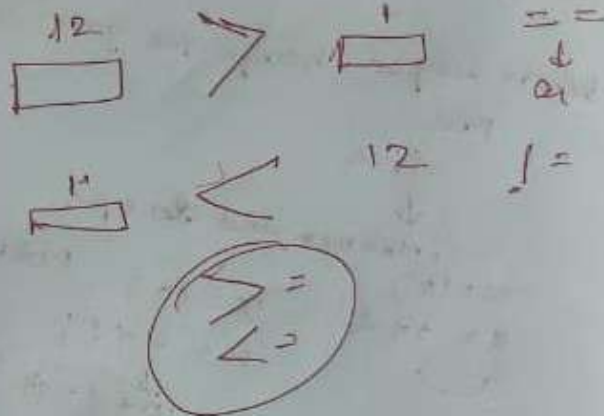
x = ++ + ++ y;
 100 + 21 y = 21

x = 101
 y = 21
 = 121

game
 yields
 for Decr

Relational operators

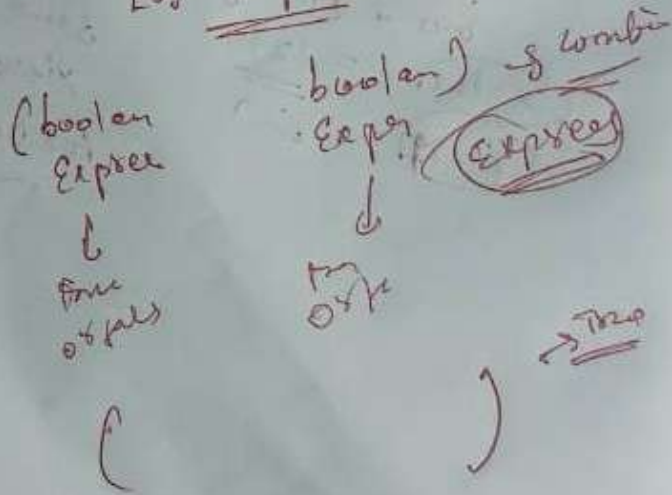
Relate value
Compare value



boolean

Logical operators

Logical Operations



If Both Condition and (88)
should be satisfied

If either of them or (19)
satisfies

If char the condition If it not (!)
as char the condition

1 - swan
0 - not swan

a b

1 0

0 1

1 1

0 0

88

0

0

1

0

a b

1 1

1 0

0 1

0 0

a || b

1

1

1

1

0

a !a

1 0

0 1

Both con -> m

11

88 H
One never
con never
par executes

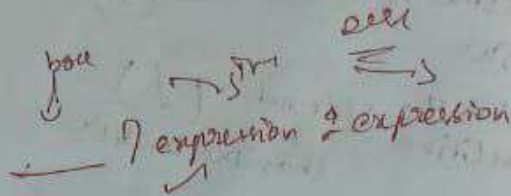
1
One
condition
True

never
executes
kill

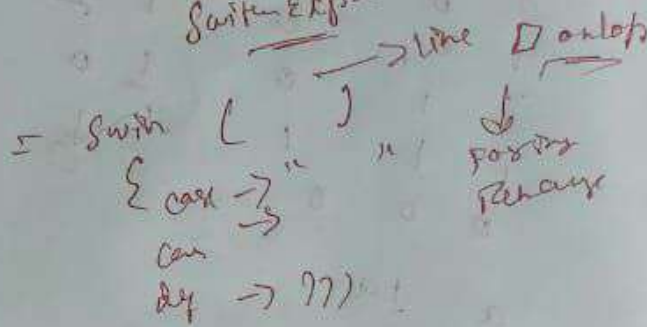
↓
short hand
express

Conditional

Operator



Switch Expression



case for -> some type -> ok
no def

One stat, one

reqd

Open All

no pa gogo

Bitwise operators

↓
bits

12

↓
bits

8	12	rem
2	6	0
2	3	0
	1	1

12 =

1100

word →

on
this

bits level

- & → and
- | → or
- ^ → xor
- ~ → not

short key
never use

<< → left side bits move (left shift)

>> → right side bit n (right shift)

mod 032

under
left
is low

Operator

Precedence

FCI -

1 / 2

+

2 > 3

4 < 5 = error

1 =

8

10

1

25

11

1

2

can us

Strings

Java strings

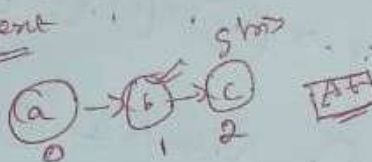
→ Sequence of unicode characters

String

String s = "";

String greet = "world" } create String

Text



Substrings

(start, end)

→ start to end - 1
to take substring from it

→ gives length

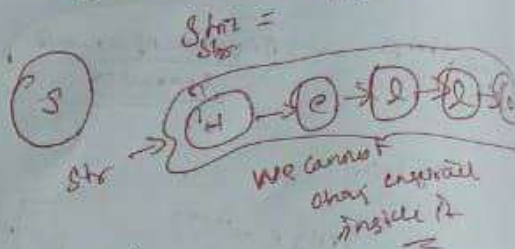
Concatenation

(+) → operator

"Hello" + "world"
→ helloworld

Immutable

"Hello"



Concatenation
new str

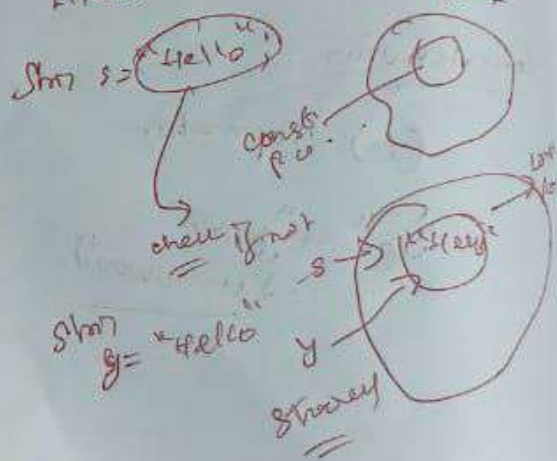
str + str 2



str =

success
new
char

Literal value are stored here



Shan's
outway - Inequal by edit

Equality

equality method

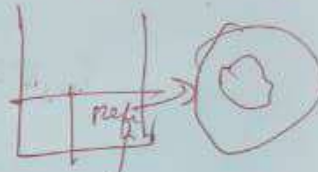
checks value

==

↓
check only if same
at first loc

Equal Ignore ans

Reference
type



like a remote
control to hold

== > both references
(if contents
are same)

Null & Empty

empty = ""
↓
object



Holds
no obj

Null $\rightarrow$ no len

lengths $\rightarrow$ X

Null pointer exception

Empty lengths = 0

✓

Code points & code values

length $\rightarrow$ number of code units

Code point count

↓
actual length

charAt(i) $\rightarrow$ code point i

offset by code point (0,1)

same (values) is
not



String APT

Build's String

String Builder

can
get A
delete
do
so

Text Block

in

in

ception

Strs Aet

Builds Strs

Strs Builder → Builds Strs from scratch

^{case}
getAt(i) → edit Str
delCharAt(i)

len() →
setLen(i)

toString() → convert to str

append()

Text Block → Java

u11 → To store space
m11 (m)

u11

Control flow

Block scope

{

}

{

→ Block of statements

}

{

}

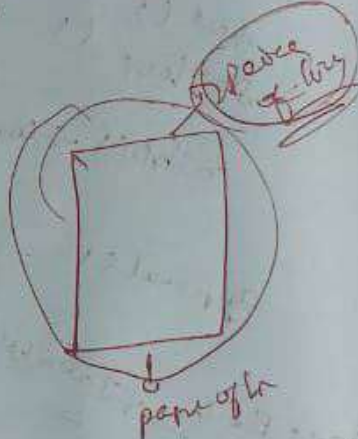
{

}

}

→ Needle in
Haystack
needle block
forbidden

Statement group
with }



Condition

if [

{

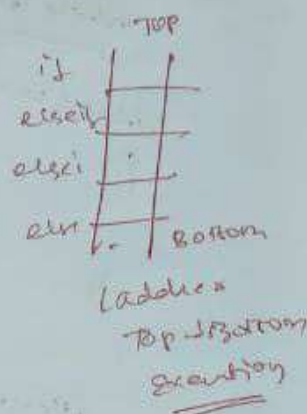
else

Conditional Statement

if [] ^{condition}
 {
 }
 boolean $\rightarrow$ true, false

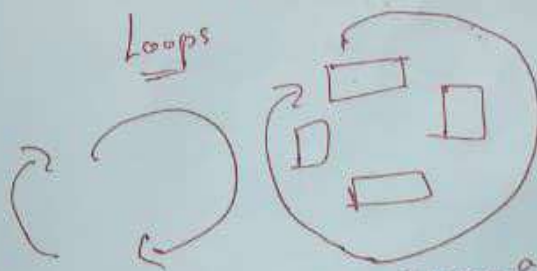
$\rightarrow$ Block
expression

else {
 $\rightarrow$ optional
 But exam
 if fails
 }



else if $\rightarrow$ can be used
 between more
 conditions

Loops



Come again & again till a status
 is reached
 Same line again & again
 until

until some condition is reached
we need to run { blocks
}

while loop
while (cond)
{
}
}

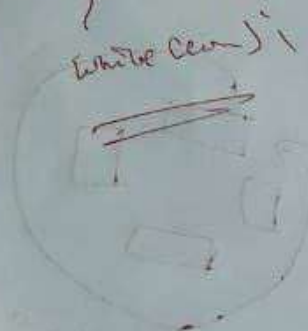
Infin loop

If never ends
condition
never
satisfies

like your
grandmother

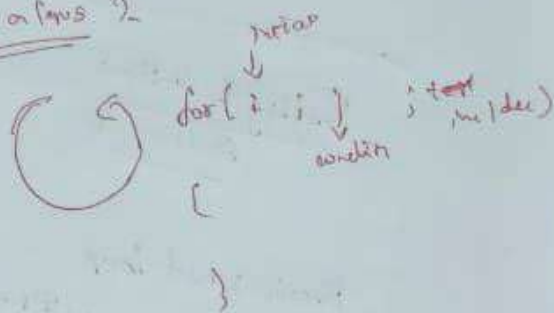
do-while

do
{
}
while (cond)



Determinate loops

Sometimes you know
How many times we should
run or loops?



Switch statement

Multiple condition based on single var

Switch
statement

```
switch ( var )  
{  
  case :  
    break;  
}
```

→ Fall through

without break
it will go
once case
match

case → Short, better, more clear
Enumeration, strongly

Java
1.5

```
switch ( )  
{  
  case → 2  
  case → 3  
  case → 4  
}
```

→ not fall thru

yield - sure mod war

Breaking & Continue
Statement

Break -> end loop

continue -> skip current iteration

When you feel you need
to end loop based
on certain condition

skip process & go
forward continue

Labelled Break & Label Continue

label : —

Label

label : —